

Predict Frontier Difficulty

A defensible portfolio of latent-reasoning tasks

The challenge is to build fair tasks, predict their difficulty before target evaluation, and revise those predictions from evidence. An impressive model implementation alone does not meet the brief. The strongest submission links each specification to a verifier, each failure to a trajectory, and each forecast to explicit assumptions.

What success actually requires

Five Harbor task packages; a preregistered `day1.md`; revised packages and a one-page `report.md` on Day 2; and reproducible evidence. At least one task must be valid and receive at most two passes in eight target-model trials. A broken environment or an unfair verifier cannot establish hardness.

Recommended strategy

Prioritize `batched-halting`, then `latent-kvcache`. Use exact gradient checks to keep the two training tasks fair. Retain `act-from-spec` as a moderate-difficulty anchor. Treat the supplied ranking as an initial hypothesis, and allocate Day 2 effort to tasks with substantive mechanism failures rather than merely low pass counts.

Three corrections to the brief

- **The portfolio is not independent.** Two tasks share ACT semantics; all five share tooling and mathematical implementation demands.
- **Behavioral symptoms are not correctness proofs.** A correct gradient implementation need not improve accuracy with more loops or keep response length in a chosen band.
- **A point forecast is insufficient.** Report a distribution over all nine outcomes, $K = 0, \dots, 8$, and test sensitivity to the proxy-to-frontier shift.

Scope and evidence. This document analyzes the supplied brief and proposes an implementation and evaluation protocol. No Harbor packages, oracle trials, pilot trials, or target-model evaluations were executed for this analysis. Numerical forecasts below are explicitly hypothetical sensitivity calculations, not experimental findings. The brief prohibits using Fable or GPT-5.6 Sol, including as authoring or critique helpers.

01 / OBJECTIVE

What the graders are measuring

The deliverable is a small benchmark-development study. Its central question is: *can you identify which realistic coding tasks will remain difficult for a stronger model, without first testing that model?* The five-task portfolio supplies both candidates and contrasts.

Separate validity from difficulty

For task j , let V_j indicate that the canonical solution works, the verifier accepts equivalent correct solutions, the specification is complete, and trials contain meaningful agent work. Let K_j be passes in eight eligible target trials. The success condition is

$$\exists j : V_j = 1 \quad \text{and} \quad K_j \leq 2.$$

A forecast of $\Pr(K_j \leq 2)$ is conditional on a valid task and a fixed evaluation setup. Keep validity evidence beside the forecast; do not treat a high predicted failure rate as evidence that the package is valid.

Fix the unit of evaluation

Specify the model identifier, agent harness, prompt, tools, time budget, sampling settings, and package hash. If both target models are evaluated, report a separate K and predictive distribution for each unless the actual assessment explicitly pools them. An unspecified mixture of models is not a stable forecasting target.

Resolve the preregistration timeline

The brief says both “before any results” and “use pilot results.” Preserve two records: an initial mechanism-based forecast before pilots, and a dated Day 1 forecast after allowed pilots but before Day 2 proxy or target results. If the evaluator literally requires a pre-pilot `day1.md`, keep that file frozen and put updates in a separate dated record. Never overwrite an old prediction after seeing outcomes.

Portfolio assessment

Task	Main uncertainty	Brief’s guess
Batched halting	Active-state and padding semantics	1/8
TBPTT gradient bug	Correct gradient boundary	2/8
Latent KV cache	Positions, masking, feedback state	3/8
GRPO length bias	Exact reduction and advantage rules	4/8
ACT from spec	Remainder and gradient precision	5/8

These counts are supplied conjectures, not measured rates. Explicit specifications may make the detach and GRPO fixes straightforward. The most promising difficulty comes from interacting invariants, particularly padding with divergent halting or latent steps with cached generation.

What would weaken the thesis?

Allowed pilots consistently construct heterogeneous tests, inspect intermediate states, and repair the intended mechanism. Such trajectories directly challenge the proposed failure story, even when an unrelated final check fails. Domain novelty alone does not establish frontier difficulty.

02 / TASKS 1 AND 5

Make halting semantics exact

A common ACT contract

For sample i , let $h_i^{(t)}$ be the state after recurrent step t , and let $p_i^{(t)} \in (0, 1)$ be its halting probability. Fix $0 < \epsilon < 1$ and a positive loop cap M . Define

$$\begin{aligned} N_i &= \min \left(\left\{ t \in \{1, \dots, M\} : \sum_{s=1}^t p_i^{(s)} \geq 1 - \epsilon \right\} \cup \{M\} \right), \\ R_i &= 1 - \sum_{s=1}^{N_i-1} p_i^{(s)}, \\ y_i &= \sum_{t=1}^{N_i-1} p_i^{(t)} h_i^{(t)} + R_i h_i^{(N_i)}, \quad \rho_i = N_i + R_i. \end{aligned}$$

The final weight is the remainder, including forced termination at M . It is not the final raw probability plus a remainder. If $N_i = 1$, then $R_i = 1$ and $y_i = h_i^{(1)}$. The weights sum to one. This is a capped task specification based on ACT [2], not a claim that all ACT implementations share identical conventions.

Task 1: batched-halting

The invariant is per-sample equivalence to isolated execution. At each loop, compute candidates and commit state, cumulative mass, output, and count only for active samples. A final-step sample receives all remaining mass. Later iterations must leave its committed state unchanged.

Padding must be excluded from attention *and* pooling for the halting head. Decide the returned values at padded positions; the cleanest contract checks valid positions and requires a stated neutral value elsewhere. Disallow all-padding samples explicitly, or define their behavior. Specify whether mask value one means valid, how positions are assigned, and whether dropout is disabled.

Tests with diagnostic value. Construct deterministic fixtures that force one sample to stop at step one and another at M ; random seeds alone do not guarantee divergence. Check y_i, N_i, R_i, ρ_i , padding-extension invariance, batch permutation, and batch-size-one equivalence. Frozen-state inspection requires an exposed trace or a public invariant; otherwise grade externally observable behavior.

Fair alternatives. Sequential isolated inference is correct under an output-only contract. Full unrolling with reconstruction is also correct when it preserves each sample's relevant pre-halt states. If speed or vectorization is required, state a measurable budget in advance; do not silently reject either alternative.

Task 5: act-from-spec

Use the same mathematical convention, but focus on a single recurrent path and differentiability. Compare outputs and gradients on fixtures whose halting decisions are safely away from the threshold. N_i is discrete; gradients are through the selected states, probabilities, and remainder, not through the threshold crossing itself. At immediate halting, the mixture may have zero halting-head gradient; that is not automatically a bug.

Portfolio implication. Shared ACT semantics reduce authoring cost but correlate failures. Keep Task 5 narrow and moderate; do not turn it into a second batching task.

03 / TASKS 2 AND 3

Gradients and cached computation

Task 2: tbptt-gradient-bug

Let $h_t = f_\theta(h_{t-1}, x)$ for $t = 1, \dots, L$, with terminal loss $\ell(h_L)$. Define $w = \min(W, L)$ for positive integer W . The intended operation is

$$\bar{h}_{L-w} = \text{stopgrad}(h_{L-w}), \quad \bar{h}_t = f_\theta(\bar{h}_{t-1}, x), \quad t = L - w + 1, \dots, L.$$

Earlier loops still determine the numerical boundary state. Only their derivative history is discarded. When $W \geq L$, retain the full unroll, including any specified trainable initializer. For $W < L$, explicitly state whether a learned initial state receives no terminal-loss gradient through the truncated path. Auxiliary losses require a separate definition.

Root cause. Detaching before every loop preserves only the final transition’s parameter contribution. Removing every detach implements full BPTT, which is also wrong when $W < L$. Shared weights make parameter-gradient comparisons more revealing than inspecting only the final state.

Verifier. Use fixed float64 parameters and batches; compare loss, final state, and every named parameter gradient for $W = 1, 2, 4$, $L = 4, 8$, plus $W \geq L$. State how disconnected gradients represented by `None` are compared with mathematical zero. Test the permitted boundary cases explicitly.

Change recommended. Keep “accuracy improves with more inference loops” as supporting evidence unless the architecture and data establish that property. Truncated-gradient correctness alone does not imply test-time compute scaling. An arbitrary accuracy margin can reject a correct repair.

Task 3: latent-kvcache

The cache is a persistent representation of *all processed prompt, latent, and emitted token positions*. A latent vector is the final-layer-normalized hidden state before the LM head, inserted through `inputs_embeds`. Specify whether this is directly in the model’s embedding space and how positional embeddings are added.

For a left-padded mask $m_{i,t}$, a useful convention is

$$\text{pos}_{i,t} = \sum_{s \leq t} m_{i,s} - 1 \quad \text{on valid positions,}$$

with padded position IDs set to zero and their keys masked. Each latent step appends one valid position; emitted tokens advance positions too. Cache slots may include physical padding while semantic position IDs count only valid entries. Attention must respect both conventions.

Resolve the off-by-one. Prefill the prompt once. Its final valid hidden state supplies the first latent input. Process exactly n_{latent} latent inputs; the resulting next-token logits choose the first greedy token. With zero latent steps, use prefill logits directly. Processing each selected token supplies the next logits. State whether return values include prediction logits, consumed embeddings, or both.

Fair cache enforcement. If caching is required, instrument processed input lengths: one prompt prefill, then one input position per incremental call. Do not require constant total step cost: ordinary attention still reads a growing cache. Compare prefill and incremental logits, tokens, positions, and cache lengths across mixed prompt lengths.

04 / TASK 4 AND VERIFICATION

Specify the objective; protect fairness

Task 4: grpo-length-bias

For one prompt, let G responses have rewards r_i , masks $m_{i,t}$, and fixed maximum response length T . Define detached advantages and policy ratios

$$A_i = r_i - \frac{1}{G} \sum_{j=1}^G r_j, \quad u_{i,t}(\theta) = \frac{\pi_{\theta}(a_{i,t} \mid s_{i,t})}{\pi_{\text{old}}(a_{i,t} \mid s_{i,t})}.$$

An explicit loss convention is

$$\mathcal{L}(\theta) = -\frac{1}{GT} \sum_{i=1}^G \sum_{t=1}^T m_{i,t} [\min(u_{i,t} A_i, \text{clip}(u_{i,t}, 1 - c, 1 + c) A_i) - \beta d_{i,t}(\theta)].$$

This removes per-response length division and group-standard-deviation scaling, the two changes relevant to the brief’s Dr. GRPO motivation [3]. It does not imply a universal guarantee that learned response length remains constant.

Unresolved contract. The supplied brief does not define $d_{i,t}$, the KL estimator. Preserve the starter’s exact expression and gradient path and publish them. One possible sampled form is $d = \exp(z) - z - 1$, where $z = \log \pi_{\text{ref}} - \log \pi_{\theta}$; choosing it is a specification decision, not something a verifier may assume silently. Fix EOS inclusion, truncated-response masks, prompt exclusion, old/reference-policy detachment, and reduction across multiple prompt groups.

Tests. Compare loss and every parameter gradient on unequal lengths, equal rewards, both advantage signs, ratios inside and outside the clipping interval, and masked tokens. Avoid exact clipping boundaries in gradient fixtures. An all-equal reward group has zero policy advantage but may still have nonzero KL loss. Reward penalties, sampling changes, and partial normalization fixes must fail.

A validity gate for every package

1. **Traceability.** Map every check to a sentence in `instruction.md`. Hidden inputs are acceptable; hidden rules are not.
2. **Positive controls.** Five fresh oracle runs must pass. A materially different correct implementation must also pass.
3. **Negative controls.** Implement each listed shortcut and verify rejection; require the starter/no-op to fail. Preserve private artifacts and result logs.
4. **Numerical contract.** Set dtype, seeds, threading, dropout, absolute and relative tolerances. If a pure absolute bound is intended, set `rtol=0`; default relative tolerances can weaken the test.
5. **Environment.** Build dependencies into the image, pin the image digest and package versions, and measure cold runtime. Five deterministic repeats check packaging consistency, not broad numerical robustness.

A private reference reduces accidental trust in edited files but is not a complete security boundary when imported alongside untrusted Python. Use trusted launch paths and, where supported, an isolated verifier with allowlisted submission files [1]. Test compatibility with the pinned Harbor version.

05 / FORECASTING

Uncertainty must remain visible

Use the brief’s model as a sensitivity model

For k passes in n comparable, eligible proxy trials,

$$p_{\text{proxy}} \sim \text{Beta}(1 + k, 1 + n - k),$$

$$\delta \sim \mathcal{N}(\mu, \sigma^2), \quad p_{\text{front}} = \text{logistic}(\text{logit}(p_{\text{proxy}}) + \delta),$$

$$\Pr(K = r \mid D) \approx \frac{1}{S} \sum_{s=1}^S \binom{8}{r} p_s^r (1 - p_s)^{8-r}, \quad r = 0, \dots, 8.$$

Use $S = 100,000$ and record the random seed. Averaging binomial probabilities directly avoids the extra noise of drawing K for each sample. Report $\mathbb{E}[K] = 8\mathbb{E}[p_{\text{front}}]$, $\Pr(K \leq 2)$, and the full histogram.

What the shift does and does not mean

A positive δ assumes that the frontier model is more likely to pass. The brief’s μ ranges are judgment calls, not calibrated estimates. Failure labels can inform the choice, but one label does not identify the shift. Report at least low, central, and high shifts; allow model-specific shifts when evaluating two targets. With no comparable pilot evidence, use an explicit prior rather than invented k/n .

Illustration: identical proxy evidence, different conclusions

The following computation assumes **hypothetical** $k = 0, n = 4, \sigma = 1$, and seed 20261008. No pilot produced these observations. Rows show probabilities, rounded to three decimals.

μ	$K = 0$	1	2	3	4	5	6	7	8
0.5	.310	.203	.145	.108	.083	.063	.045	.030	.014
1.0	.230	.176	.142	.118	.100	.084	.069	.052	.029
2.0	.114	.109	.108	.110	.112	.116	.119	.117	.094

The corresponding $\Pr(K \leq 2)$ values are 0.657, 0.547, and 0.332; expected pass counts are 2.05, 2.65, and 3.97. The transfer assumption can reverse a hardness judgment despite unchanged proxy evidence.

Correct the “zero of eight” interpretation

With zero passes in eight trials and a uniform prior, $p \mid D \sim \text{Beta}(1, 9)$. Its upper one-sided 95% credible bound is 0.283; its equal-tailed 95% interval is approximately $[0.0028, 0.3363]$. “About 30%” is a rough summary, not an interchangeable frequentist confidence interval or a frontier estimate.

Dependence and selection

Repeated trials share harnesses, and different proxy models need not share a pass rate. Stratify results or use a hierarchical model rather than blindly pooling. For the portfolio, $1 - \prod_j (1 - q_j)$ assumes independence, where $q_j = \Pr(K_j \leq 2 \mid V_j = 1)$. Without that assumption, the union probability lies between $\max_j q_j$ and $\min(1, \sum_j q_j)$. Freeze a new package version after hardening and collect fresh evidence; do not attach old pass rates to a changed task.

06 / EXECUTION

A realistic two-day workflow

Day 1: establish validity, then collect evidence

Time budget	Concrete output and stopping rule
Hours 0–1	Check assessment rules; freeze initial forecasts; pin Harbor and model allowlist; establish shared environment and reward contract.
Hours 1–4	Complete batched-halting end to end. Require oracle, shortcut, alternate-solution, and specification checks before reuse.
Hours 4–7	Build cache task and narrow ACT anchor; implement gradient-based TBPTT and GRPO verifiers with fixed fixtures.
Hours 7–10	Audit all five packages, inspect starter behavior, measure runtime, and launch 3–4 allowed pilots per task in parallel where capacity permits.
Hours 10–12	Read trajectories, record failures, compute forecast sensitivity, and freeze the Day 1 submission before held-out outcomes.

This is an allocation plan, not a claim that five validated packages fit reliably into twelve hours. Pilot time is 15–20 trajectories plus review; obtain credentials and estimate wall time early. Reuse infrastructure, not tests that accidentally share the same reference bug. If validity work overruns, reduce implementation scope before reducing fairness checks.

Day 2: repair validity before increasing difficulty

Label every returned run: **INFRA**, **UNFAIR**, **SHALLOW**, **MECHANISM**, or **PASS**. Keep factual status separate from the interpretive label: a shallow but legitimate failure may remain in the pass-rate denominator, although it is weak transfer evidence. Exclude infrastructure-invalid trials under a recorded policy and replace them to obtain the required eligible sample; never relabel a difficult legitimate trial as infrastructure merely because it failed.

- **Any UNFAIR:** fix the specification or verifier and rerun full validation. Invalidate affected historical comparisons.
- **Zero or one pass, substantive mechanism failures:** preserve the task and invest in evidence quality.
- **Zero passes, mostly shallow failures:** improve setup clarity; do not infer frontier hardness.
- **Three or more passes:** inspect the success strategy. Harden only through a coherent, explicit interaction, then version and repilot; otherwise keep it as a calibration task.

Concentrate remaining time on the best one or two valid candidates. Adding a second bug is useful only when it tests a meaningful interaction and remains clearly specified.

Make trajectories auditable

For each run, store task hash, model and harness versions, seed/settings, reward, eligibility, label, step indices, and artifact paths. A mechanism claim needs the actual action: for example, “step 27 tested only equal-depth examples; step 31 stopped; private check showed halted accumulator drift.” That sentence is an illustration, not observed evidence. Include successful counterexamples that weaken the theory.

07 / SUBMISSION

What to submit and how to defend it

Package contents

Each task needs `task.toml`, `instruction.md`, a reproducible environment, starter repository, private tests, and the canonical solution. Use the schema generated by the pinned Harbor installation rather than copying a possibly outdated configuration. Harbor documents `/logs/verifier/reward.txt` and gives `reward.json` precedence when both exist [1]; choose one binary reward channel and avoid stale competing files. Record diagnostic status separately so a timeout cannot silently become evidence of model weakness.

The Day 1 forecast record

Use one row per task: rank; exact package version; predicted K summary and full distribution; mechanism; pilot k/n ; trajectory evidence; transfer assumptions; validity controls; and authoring workflow. Preserve prompts and tool settings. Report “not run” when evidence is missing. Do not claim a separate critique agent or independent review unless it actually occurred.

The one-page Day 2 report

Reserve roughly a quarter page for changes, a quarter for pass rates and mechanism evidence, a quarter for the five predictive histograms, and a quarter for the hardest-task claim, falsifier, and workflow. Keep large logs in `evidence/`. For each revision, state what changed, why, which trial motivated it, and whether previous results still apply.

A defensible falsifier is specific: “If the target routinely tests divergent-depth, padded batches and still passes at least four of eight trials, the proposed early-stopping mechanism is weaker than forecast.” A single pass is not a falsifier for a probabilistic claim; a repeated successful strategy can be.

Final release gate

- Five packages build offline at verification time; canonical and alternate solutions pass; the starter and all listed shortcuts fail.
- Every scored condition is public in the task specification; numerical tolerances and gradient semantics are explicit.
- Forecast files are timestamped, distributions sum to one, and changed versions are distinguishable.
- Every reported trial has logs, eligibility, and a justified label; no prohibited target model was used in the workflow.
- The report distinguishes observed results, subjective transfer assumptions, and untested hypotheses.

Research context: keep only what supports the design

The long state-of-the-art survey is optional context, not the argument for difficulty. ACT supports the halting specification [2]; the GRPO paper motivates an objective-level bug [3]; Coconut provides a concrete continuous-hidden-state feedback mechanism [4]. The claims about other 2025–2026 systems in the supplied brief were not independently audited here. Avoid repeating broad benchmark superiority or hardware-efficiency claims without checking the particular experiment and hardware assumptions.

Sources

[1] Harbor Framework. *Task Structure*, official repository documentation. [Harbor task documentation](#). Accessed 8 October 2026. Pin a release for implementation.

[2] A. Graves. *Adaptive Computation Time for Recurrent Neural Networks* (2016). [arXiv:1603.08983](#).

[3] Z. Liu et al. *Understanding R1-Zero-Like Training: A Critical Perspective* (2025). [arXiv:2503.20783](#).

[4] S. Hao et al. *Training Large Language Models to Reason in a Continuous Latent Space* (2024). [arXiv:2412.06769](#).

Primary input: the user’s pasted “Predict Frontier Difficulty” agent brief. All uncited recommendations and mathematical analysis are this document’s assessment.